

# GPU Server System Architecture

## GPU 서버 시스템 아키텍처

GPU 서버 성능은 GPU 자체가 아니라 CPU, 메모리, PCIe, NUMA, NIC, NVLink/NVSwitch의 연결 구조 전체에 의해 결정된다.

### GPU Server란

GPU는 단독으로 동작하지 않음. 서버 내부의 CPU, Host Memory, PCIe 버스, NIC, 스토리지와 연결되어 동작하는 가속 장치임. GPU 자체 성능이 아무리 높아도 데이터 경로에 병목이 있으면 전체 성능이 떨어짐.

따라서 GPU 서버를 이해할 때는 **GPU가 어디에 연결되어 있고, 어떤 경로로 데이터를 주고 받는지를** 함께 봐야

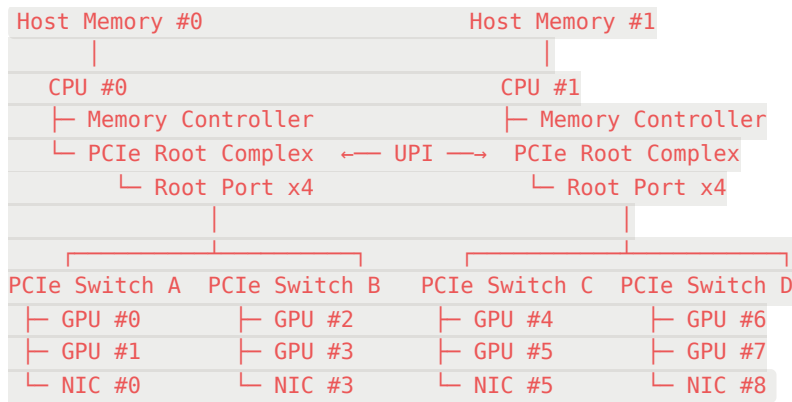
### 주요 구성요소

| 컴포넌트              | 위치              | 역할                                   |
|-------------------|-----------------|--------------------------------------|
| CPU Core          | CPU 칩 내부        | 프로그램 실행, 작업 제어, GPU 작업 요청            |
| Memory Controller | CPU 칩 내부        | CPU Core와 DRAM 연결 (별도 메모리 버스 사용)     |
| Host Memory       | CPU 외부 DRAM     | CPU가 사용하는 시스템 메모리                    |
| PCIe Root Complex | CPU 칩 내부        | CPU와 PCIe 버스를 연결하는 최상위 인터페이스         |
| PCIe Root Port    | Root Complex 내부 | PCIe 트리가 시작되는 논리적 경계점                |
| PCIe Switch       | CPU와 디바이스 사이    | GPU, NIC, NVMe 등을 연결하기 위한 PCIe 확장 장치 |
| GPU               | PCIe Switch 하위  | 대량 병렬 연산 가속 장치                       |
| NIC               | PCIe Switch 하위  | 네트워크 송수신 장치                          |

PCIe는 GPU/NIC/NVMe 같은 장치 연결용이고, DRAM 접근은 Memory Controller를 통한 별도 메모리 버스로 이루어짐.

### Dual-CPU GPU 서버 구조

고성능 GPU 서버는 대부분 Dual-CPU 구조를 사용. 각 CPU는 자기 메모리, 자기 PCIe Root Complex, 자기 디바이스를 가지며, CPU 간에는 UPI로 연결됨.



CPU #0에 연결된 GPU #0~#3, NIC #0~#4 → NUMA Node 0

CPU #1에 연결된 GPU #4~#7, NIC #5~#9 → NUMA Node 1

## NUMA (Non-Uniform Memory Access)

멀티 CPU 시스템에서 CPU가 메모리에 접근하는 속도가 메모리 위치에 따라 달라지는 구조.

| 접근 유형         | 경로                                                       | 성능 |
|---------------|----------------------------------------------------------|----|
| Local Access  | CPU #0 → 자기 Memory Controller → Host Memory #0           | 빠름 |
| Remote Access | CPU #0 → UPI → CPU #1 Memory Controller → Host Memory #1 | 느림 |

**NUMA Node = 하나의 CPU 소켓 + 그 CPU에 직접 연결된 모든 것의 묶음** (CPU Core, Cache, Memory Controller, Host Memory, PCIe Root Complex, PCIe Device)

## 주요 데이터 패스

| 데이터 패스                | 경로                                                        | 특징                     |
|-----------------------|-----------------------------------------------------------|------------------------|
| CPU → Local Memory    | Core → Memory Controller → DRAM                           | 가장 빠름                  |
| CPU → Remote Memory   | Core → UPI → 상대 CPU MC → DRAM                             | 레이턴시 증가, 대역폭 감소        |
| GPU → Host Memory     | GPU → PCIe Switch → Root Port → Root Complex → MC → DRAM  | GPU-CPU 간 기본 이동 경로     |
| GPU → NIC (같은 Switch) | GPU → PCIe Switch → NIC                                   | 가까움, GPUDirect RDMA 최적 |
| GPU → NIC (다른 NUMA)   | GPU → Switch → CPU #0 RC → UPI → CPU #1 RC → Switch → NIC | 먼 경로, 성능 저하 큼          |

| 데이터 패스                | 경로                          | 특징            |
|-----------------------|-----------------------------|---------------|
| GPU → GPU<br>(NVLink) | GPU → NVLink/NVSwitch → GPU | PCIe 우회 고속 통신 |

성능은 홑 수에 비례하여 저하됨. 특히 UPI를 경유하는 Remote NUMA 통신이 가장 큰 병목.

## PCIe / NVLink / NVSwitch 비교

| 구분       | 주 용도                | 통신 대상                               | 특징               |
|----------|---------------------|-------------------------------------|------------------|
| PCIe     | CPU 중심 장치 연결 버스     | CPU ↔ GPU, CPU ↔ NIC,<br>CPU ↔ NVMe | GPU 인식·제어의 기본 경로 |
| NVLink   | GPU 간 고속 직접 통신      | GPU ↔ GPU                           | PCIe보다 대역폭 훨씬 높음 |
| NVSwitch | 여러 GPU의 NVLink를 스위칭 | GPU 여러 대 간 All-to-All               | 대규모 학습에서 핵심      |

GPU가 NVLink를 사용하더라도, 서버가 GPU를 인식·제어하는 기본 경로는 항상 PCIe임.

## NVLink 세대별 대역폭

| 세대    | 대표 GPU         | 총 대역폭      | Line 수 | Line당 대역폭 |
|-------|----------------|------------|--------|-----------|
| Gen 4 | Hopper H100    | 900 GB/s   | 18     | 50 GB/s   |
| Gen 5 | Blackwell B200 | 1,800 GB/s | 18     | 100 GB/s  |
| Gen 6 | Rubin          | 3,600 GB/s | 36     | 100 GB/s  |

## nvidia-smi topo -m 라벨 해석

| Label | 의미                               | 성능 순위          |
|-------|----------------------------------|----------------|
| NV#   | GPU 간 NVLink #개 연결               | 최고 (GPU ↔ GPU) |
| PIX   | PCIe Bridge 1개 경유 (같은 Switch 하위) | 최고 (GPU ↔ NIC) |
| PXB   | PCIe Bridge 2개 이상 경유             | 보통             |
| NODE  | 같은 NUMA Node, Host Bridge 경유     | 느림             |
| SYS   | 다른 NUMA Node, UPI 경유             | 가장 느림          |

성능 순서: NV# > PIX > PXB > NODE > SYS

- GPU ↔ GPU: NV# 관계가 최적 (NVLink/NVSwitch)

- GPU ↔ NIC: PIX 관계가 최적 (같은 PCIe Switch)

## GPUDirect RDMA 최적 GPU-NIC 매핑

| GPU   | 최적 NIC | 관계  | PCIe Switch | NUMA Node |
|-------|--------|-----|-------------|-----------|
| GPU 0 | mlx5_0 | PIX | Switch A    | Node 0    |
| GPU 1 | mlx5_2 | PIX | Switch A    | Node 0    |
| GPU 2 | mlx5_3 | PIX | Switch B    | Node 0    |
| GPU 3 | mlx5_4 | PIX | Switch B    | Node 0    |
| GPU 4 | mlx5_5 | PIX | Switch C    | Node 1    |
| GPU 5 | mlx5_7 | PIX | Switch C    | Node 1    |
| GPU 6 | mlx5_8 | PIX | Switch D    | Node 1    |
| GPU 7 | mlx5_9 | PIX | Switch D    | Node 1    |

아무 NIC나 사용하면 되는 게 아니라, **GPU와 같은 PCIe Switch에 있는 NIC를 선택**해야 RDMA 성능이 최적.

## 실무 튜닝 체크리스트

| 확인 항목                       | 이유                       |
|-----------------------------|--------------------------|
| GPU가 어느 NUMA Node에 속하는지     | 가까운 CPU와 메모리 사용을 위해      |
| GPU-NIC 관계가 PIX인지 SYS인지     | RDMA 성능과 네트워크 처리량에 직결    |
| GPU 간 NVLink/NVSwitch 연결 여부 | 멀티 GPU 학습 통신 성능에 직결      |
| CPU Affinity가 맞는지           | Remote NUMA 접근 방지        |
| Host Memory-GPU 간 경로        | 데이터 로딩 병목 확인             |
| PCIe Switch 배치              | GPU-NIC, GPU-NVMe 경로 최적화 |



GPU 서버 성능 최적화의 본질은 **모든 구성요소를 가장 가까운 경로로 연결해서 사용하는 것이다**. GPU와 NIC는 같은 PCIe Switch(PIX)로, 프로세스는 해당 GPU와 같은 NUMA Node CPU에 바인딩하고, GPU 간 통신은 NVLink/NVSwitch를 활용한다. `nvidia-smi topo -m` 으로 구조를 확인하고, **홑이 적은 경로로 데이터를 흘리는 것이 핵심**.